EXECUTIVE SUMMARY

How should we decide who gets a loan?

Recommendation: use a 0.40 decision threshold, pending further validation

Not the textbook default of 0.50. This threshold was chosen using a validation set and confirmed on data the model never touched -- an honest, out-of-sample estimate of a $323,413 profit improvement, not an optimistic one.

AUC 0.91	0.40	$9,993	+$323K
Model quality (test)	Selected threshold	Avg loss, one missed bad loan	Honest gain from tuning

Why not just use the most accurate model?

At the standard 0.50 threshold, the model is actually MORE accurate (91.9%) than at our recommended 0.40 (91.1%). But 0.50 also approves more bad customers (458 vs. 389) in exchange for rejecting fewer good ones (69 vs. 191) -- and each missed bad customer costs the bank roughly 10x more than each wrongly-rejected good one. The more accurate threshold is not the more profitable one. A model optimized purely for accuracy would have picked the wrong cutoff for the bank's actual goal.

What we did differently this time

An earlier version of this analysis selected the threshold by testing it directly on the same data used to report results -- a classic methodology risk that can make the reported gain look better than it truly is. This version corrects that: the threshold is chosen using a separate validation set, then applied exactly once to data the model has never influenced any decision with, to produce a trustworthy, out-of-sample number.

Source: Kaggle Credit Risk Dataset (laotse) | Analysis: 2026-09

DATA QUALITY

A hidden bias we almost missed

Before any modeling, a routine data-cleaning step nearly introduced a serious, invisible bias into this analysis. We think this is worth explaining plainly, because it's the kind of mistake that is easy to make and hard to catch afterward.

The problem

76% of the interest-rate values in the raw data were unreadable -- corrupted text instead of numbers.

The cause

We traced this to a specific, fixable-in-hindsight error: the file had previously been opened and re-saved in Excel with Russian regional settings, which silently converted decimal numbers (like 16.02) into calendar dates. The original values could not be recovered, so they were replaced with estimates instead (next step).

The near-mistake

Our first instinct was to simply discard the corrupted rows and analyze the rest. But checking who was affected revealed the corruption was NOT random: grade-A loans (98.2% missing) and grade-B loans (98.0% missing) -- our safest customers -- were almost entirely affected, versus 24-43% for riskier grades. Discarding rows would have removed almost all of our safest customers, making the entire dataset look far riskier than reality.

The fix

Instead of discarding data, we filled each gap using the typical interest rate for that customer's own credit grade -- preserving every customer while avoiding the bias.

Verification

We compared outcomes for imputed vs. original data within each credit grade. They were reasonably close, which is evidence the imputation did not materially distort the default pattern within grade -- though it remains an estimate, not a recovery of the true original values.

Why we're telling you this

It's a concrete example of how an unglamorous data-quality check can be the difference between a trustworthy model and a quietly biased one.

MODEL STRATEGY & NEXT STEPS

Simpler beat more complex

We tested a more advanced technique -- teaching the model itself about the cost difference between mistakes, not just adjusting the decision threshold afterward. Surprisingly, it did not help, and this held true even after correcting the evaluation methodology described on page 1.

Approach	Test Profit	vs. best
A: Standard model + tuned threshold	**$1,449,533**	**Best**
B2: Cost-aware training (moderate)	$1,257,010	-$192,523
B1: Cost-aware training (full strength)	$1,220,395	-$229,138

What this means

The cost information was already being used once, in choosing the 0.40 threshold. Adding it a second time, into the model's training, over-corrected and made the model's risk-ranking worse -- and the more aggressively we applied it, the worse the result got. This is a useful, generalizable finding: added modeling complexity should be verified to actually help before it is adopted, not assumed to help.

Recommended next steps

1. Use the standard model with the 0.40 decision threshold for further validation; do not add cost-sensitive training.
2. Replace illustrative assumptions (10% recovery rate) with the bank's actual figures before using this for a live budget or policy decision -- this assumption directly scales every profit figure in this report.
3. Treat grade G (98% observed default, only 64 real customers) as an auto-reject policy rather than a model-driven decision -- the sample is too small and extreme to trust threshold tuning there.
4. Before production use: run a fairness audit, add explainability for individual decisions, and validate on more recent (out-of-time) data -- see the full workbook's Limitations sheet for the complete list.